

Мастер-пром프트 для разработки Telegram Mini App ВКРАЙНОСТИ

Ты senior full-stack / frontend developer. Твоя задача — разработать Telegram Mini App для проекта **ВКРАЙНОСТИ**, связанный с текущим сайтом и полностью синхронизированный с его данными.

1. Доступ к проекту

Работай с файлами сайта по локальному адресу:

```
C:\Users\HP\Documents\Cursor projects\Commercial\Vkrainosti
```

Ты имеешь право свободно исследовать структуру проекта, читать компоненты, стили, данные, endpoints, конфигурации, текущую логику загрузки туров, расписания, медиа и заявок.

Перед началом разработки обязательно проведи аудит проекта:

1. Определи стек проекта.
2. Найди структуру страниц и компонентов.
3. Найди, откуда сайт получает данные туров.
4. Найди существующие endpoints / JSON / S3-ссылки / GAS-интеграции.
5. Найди, как сайт подтягивает медиа.
6. Найди, как реализована сезонная навигация.
7. Найди, как реализованы карточки туров и страницы туров.
8. Определи визуальный стиль сайта: цвета, шрифты, отступы, карточки, кнопки, анимации, сетки, настроение бренда.

Не начинай писать код, пока не поймёшь текущую архитектуру.

2. Главная цель

Создать **Telegram Mini App**, который является компактным Telegram-зеркалом актуальных туров сайта.

Mini App должен:

- открываться внутри Telegram;
- использовать визуальный стиль сайта ВКРАЙНОСТИ;
- показывать все туры, которые уже отображаются на сайте;
- использовать тот же источник данных, что и сайт;
- не требовать изменений в текущей Google Sheets таблице;
- не вводить отдельную логику публикации для Telegram;
- не создавать отдельную CMS;
- не дублировать данные вручную;
- использовать существующие endpoints / JSON / S3 / GAS-интеграции сайта;

- позволять пользователю быстро выбрать тур и оставить заявку.

Главный принцип:

Всё, что отображается на сайте, отображается и в Telegram Mini App.

3. Что не нужно делать

Не добавляй новые поля в Google Sheets.

Не меняй структуру текущей таблицы.

Не создавай отдельную логику:

```
showInTelegram  
telegramStatus  
telegramOnly  
miniAppOnly
```

Не создавай отдельный backend, если можно использовать существующую систему.

Не хардкодь туры, сезоны, медиа, ссылки, цены, даты и описания.

Не создавай большие массивы моковых данных внутри компонентов.

Не создавай спагетти-код.

Не усложняй архитектуру.

Не делай личный кабинет.

Не делай историю заявок.

Не делай подбор тура.

Не делай оплату.

Не делай систему бонусов.

Не делай отдельную админку.

4. Утверждённый функционал Mini App

Главный экран

Главный экран должен быть каталогом туров.

Структура:

ВКРАЙНОСТИ

Туры по Приморью

[Зима] [Весна] [Лето] [Осень]

Все туры выбранного сезона:

Карточка тура

Карточка тура

Карточка тура

Функционал:

- загрузка туров из существующего источника данных сайта;
- переключение сезонов как на сайте;
- фильтрация туров по сезону;
- отображение карточек туров;
- переход на экран конкретного тура.

На главном экране не должно быть:

Подобрать тур

Мои заявки

Личный кабинет

Карточка тура

Карточка тура должна быть компактной и удобной для мобильного Telegram-интерфейса.

Рекомендуемая структура:

[Главное фото]

Название тура

Длительность · Сложность · Цена

Ближайшая дата / сезон / краткая информация

[Подробнее]

Карточка должна использовать реальные данные сайта.

Нельзя хардкодить текст карточек.

Экран тура

Экран тура должен быть короткой мобильной версией страницы тура сайта.

Структура:

Главное фото
Название
3–5 ключевых параметров
Ближайшие даты
Короткое описание
Галерея
Кнопка “Оставить заявку”

Также можно добавить вторичную кнопку:

Открыть полное описание на сайте

Эта кнопка должна вести на соответствующую страницу тура на сайте, если такая ссылка доступна в данных.

Не нужно переносить всю длинную SEO-страницу сайта в Mini App.

Mini App должен быть быстрым сценарием выбора и заявки, а не полной заменой сайта.

Экран заявки

Форма заявки:

Выбранный тур
Выбранная дата
Имя
Телефон
Количество участников
Комментарий
Кнопка “Отправить”

Telegram Mini App должен использовать простую Telegram-привязку пользователя.

При открытии Mini App нужно получить доступные Telegram-данные пользователя:

```
telegram_id  
username  
first_name  
last_name  
language_code
```

Эти данные нужно передавать вместе с заявкой, если технически доступно.

Телефон Telegram автоматически не считать обязательным источником данных. Пользователь должен ввести телефон вручную, если текущая система заявки требует телефон.

5. Данные и синхронизация

Текущая Google Sheets таблица не меняется.

Существующий GAS-код остаётся источником логики отображения.

Mini App должен использовать те же данные, что и сайт:

```
Google Sheets  
  ↓  
существующий GAS-код  
  ↓  
готовые JSON / endpoints / S3-данные  
  ↓  
сайт  
  ↓  
Telegram Mini App
```

Твоя задача — найти, как сайт сейчас получает данные, и подключить Mini App к тому же источнику.

Если сайт использует:

```
tours_list.json  
schedule.json  
S3-хранилище  
GAS endpoint  
локальные JSON-файлы
```

то Mini App должен переиспользовать эту же систему.

Нельзя создавать отдельный источник данных для Mini App, если в этом нет крайней необходимости.

6. Медиа

Mini App обязан переиспользовать существующую систему медиа сайта.

Нужно найти:

- откуда сайт берёт изображения;
- как формируются ссылки на изображения;
- используется ли S3;
- есть ли helper-функции для URL медиа;
- есть ли fallback-логика;
- как устроены галереи туров.

Mini App должен подтягивать изображения и галереи из той же системы.

Не копируй изображения вручную.

Не создавай отдельные папки с дублирующими медиа.

Не хардкодь S3-ссылки внутри компонентов, если в проекте уже есть функция или слой для работы с медиа.

7. Визуальный стиль

Mini App должен ощущаться как часть сайта ВКРАЙНОСТИ.

Перед реализацией изучи:

- цвета сайта;
- typography scale;
- кнопки;
- карточки;
- border-radius;
- тени;
- фоновые изображения;
- сезонные блоки;
- UI карточек туров;
- mobile-адаптацию;
- настройку бренда;
- паттерны отступов.

Задача — не скопировать сайт 1-в-1, а адаптировать его под Telegram Mini App.

Сайт — это большая витрина.

Mini App — это быстрый мобильный сценарий.

Интерфейс должен быть:

- компактным;
- удобным с телефона;
- быстрым;
- читаемым;
- визуально связанным с сайтом;
- без перегруженных лендинговых секций.

8. Telegram Mini App логика

Добавь аккуратную интеграцию с Telegram WebApp API.

Нужно поддержать:

- получение данных Telegram-пользователя;
- корректную инициализацию Mini App;
- адаптацию под Telegram viewport;
- безопасное поведение вне Telegram, если приложение открыто в браузере во время разработки;
- fallback-режим для локальной разработки.

Примерная логика:

```
Если приложение открыто в Telegram:  
  использовать window.Telegram.WebApp  
  получить initDataUnsafe.user  
  передать telegram user data в форму заявки
```

```
Если приложение открыто вне Telegram:  
  не ломать приложение  
  использовать dev fallback
```

Не завязывай весь интерфейс жёстко на Telegram API так, чтобы он не открывался локально.

9. Архитектурные требования

Разрабатывай на senior-level, но без переусложнения.

Главные правила:

1. Простое решение лучше сложного.
2. Не создавать лишние абстракции.
3. Не дублировать бизнес-логику.
4. Не хардкодить данные.
5. Не смешивать UI, data fetching и business logic в одном файле.
6. Не создавать огромные компоненты.

7. Не создавать массивы моковых туров в production-коде.
8. Не ломать существующий сайт.
9. Не менять текущую таблицу.
10. Не менять текущий GAS без необходимости.
11. Все изменения должны быть понятными и обратимыми.

Код должен быть:

- модульным;
- читаемым;
- типизированным, если проект использует TypeScript;
- совместимым с текущим стеком;
- аккуратно встроенным в проект;
- без глобальных побочных эффектов;
- с понятными именами файлов и функций.

10. Рекомендуемая структура

Сначала изучи проект и выбери структуру, которая лучше всего подходит текущему стеку.

Если проект на Next.js, можно рассмотреть структуру:

```
src/app/telegram/  
  page.tsx  
  
src/components/telegram/  
  TelegramMiniApp.tsx  
  SeasonTabs.tsx  
  TelegramTourCard.tsx  
  TelegramTourDetails.tsx  
  TelegramRequestForm.tsx  
  
src/lib/telegram/  
  telegramWebApp.ts  
  getTelegramUser.ts  
  
src/lib/tours/  
  getTours.ts  
  normalizeTours.ts
```

Но не создавай эту структуру механически.

Сначала проверь, как уже организован проект, и вступи в существующий стиль.

11. Работа с сезонами

Mini App должен использовать сезоны как на сайте.

Нужно найти существующие значения сезонов в данных сайта.

Не придумывай новые значения, если уже есть старые.

Нужно аккуратно нормализовать сезоны только на уровне отображения, если это нужно.

Пример интерфейса:

Зима | Весна | Лето | Осень

Фильтрация должна происходить по данным из существующего источника.

12. Работа с заявками

Изучи, как сейчас сайт отправляет заявки.

Если уже есть endpoint / GAS / Telegram bot / форма заявки, используй существующую систему.

Форма Mini App должна передавать:

```
tour_id / tour_title
selected_date
name
phone
participants_count
comment
telegram_id
telegram_username
telegram_first_name
telegram_last_name
source = telegram-mini-app
```

Если текущий endpoint не принимает часть этих полей, не ломай его.

Сначала определи совместимый формат.

Если нужно минимальное расширение отправки заявки — предложи самый простой способ, но не меняй структуру системы без необходимости.

13. Проверка и качество

После реализации проверь:

1. Mini App открывается как обычная страница в браузере.
2. Mini App не ломается без Telegram API.
3. Список туров подтягивается из текущего источника.

4. Сезоны переключаются корректно.
 5. Отображаются только те туры, которые уже есть на сайте.
 6. Медиа подтягиваются через существующую систему.
 7. Карточка тура открывается.
 8. Экран тура отображает реальные данные.
 9. Форма заявки валидирует обязательные поля.
 10. Заявка отправляется в существующую систему или подготовленный endpoint.
 11. Telegram user data добавляется к заявке, если доступна.
 12. Ничего не сломано на основном сайте.
 13. Нет хардкода туров.
 14. Нет новых колонок в таблице.
 15. Нет отдельной логики публикации для Telegram.
-

14. Что нужно выдать в результате

После завершения работы предоставь:

1. Краткое описание, что было реализовано.
 2. Список изменённых файлов.
 3. Как запустить Mini App локально.
 4. Где находится страница Mini App.
 5. Какие endpoints используются.
 6. Как Mini App получает туры.
 7. Как Mini App получает медиа.
 8. Как работает Telegram-привязка.
 9. Как работает отправка заявки.
 10. Что осталось на следующий этап.
-

15. Приоритет разработки

Работай в таком порядке:

Этап 1. Исследование

- изучить проект;
- найти источники данных;
- найти медиа-логику;
- найти стили сайта;
- найти форму заявки;
- найти сезонную структуру.

Этап 2. План

Сформировать короткий технический план:

что переиспользуем;

что создаём новое;

какие файлы меняем;
какие endpoints используем;
какие риски есть.

Этап 3. Реализация MVP

- создать маршрут / страницу Mini App;
- подключить данные туров;
- реализовать сезонные tabs;
- реализовать список карточек;
- реализовать экран тура;
- реализовать форму заявки;
- подключить Telegram user data;
- подключить отправку заявки.

Этап 4. Проверка

- проверить локально;
- проверить responsive;
- проверить fallback без Telegram;
- проверить отсутствие хардкода;
- проверить, что сайт не сломан.

16. Главная формулировка задачи

Создай Telegram Mini App для ВКРАЙНОСТИ как компактную Telegram-витрину туров, полностью синхронизированную с текущим сайтом. Приложение должно использовать существующую Google Sheets + GAS + S3 / endpoints систему сайта, не менять таблицу, не вводить отдельную логику публикации, сохранять визуальный стиль сайта и позволять пользователю быстро выбрать сезон, открыть тур и оставить заявку с простой Telegram-привязкой пользователя.

Работай как senior developer: просто, чисто, модульно, без хардкода, без лишней архитектуры и без спагетти-кода.